

> Linear Regression

↳ 4 ô bị ẩn

> Gradient Descent

↳ 2 ô bị ẩn

> Perceptron

↳ 5 ô bị ẩn

> Logistic Regression

↳ 9 ô bị ẩn

> AR Face (LR)

↳ 29 ô bị ẩn

> Softmax Regression

↳ 20 ô bị ẩn

> Softmax Regression cho MNIST

↳ 15 ô bị ẩn

> Softmax Regression cho MNIST dự đoán nhiều số

↳ 7 ô bị ẩn

> Multi Layer Perceptron

↳ 17 ô bị ẩn

> MNIST_CNN

↳ 7 ô bị ẩn

✓ Fashion MNIST CNN tensorflow

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPool2D, Flatten, Dense
import pandas as pd
```

```
# Load and preprocess the MNIST dataset
```

```
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
```

```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ————— 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ————— 0s 0us/step

```

```

print("X_train: " + str(X_train.shape))
print("y_train: " + str(y_train.shape))
print("X_test: " + str(X_test.shape))
print("y_test: " + str(y_test.shape))

```

```

X_train: (60000, 28, 28)
y_train: (60000,)
X_test: (10000, 28, 28)
y_test: (10000,)

```

```

X_train = X_train / 255.0
X_test = X_test / 255.0

```

```

#defining model
model=Sequential()
#adding convolution layer
model.add(Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)))
#adding pooling layer
model.add(MaxPool2D(2,2))
#adding fully connected layer
model.add(Flatten())
model.add(Dense(100,activation='relu'))
#adding output layer
model.add(Dense(10,activation='softmax'))

```

```

#compiling the model
model.compile(loss='sparse_categorical_crossentropy',optimizer='adam',metrics=['accuracy'])
#fitting the model
model.fit(X_train,y_train,epochs=10)

```

```

Epoch 1/10
1875/1875 ————— 40s 21ms/step - accuracy: 0.8130 - loss: 0.5246
Epoch 2/10
1875/1875 ————— 39s 21ms/step - accuracy: 0.8993 - loss: 0.2755
Epoch 3/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9181 - loss: 0.2246
Epoch 4/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9314 - loss: 0.1863
Epoch 5/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9430 - loss: 0.1577
Epoch 6/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9480 - loss: 0.1388
Epoch 7/10
1875/1875 ————— 37s 20ms/step - accuracy: 0.9566 - loss: 0.1160
Epoch 8/10
1875/1875 ————— 41s 20ms/step - accuracy: 0.9652 - loss: 0.0961
Epoch 9/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9724 - loss: 0.0786
Epoch 10/10
1875/1875 ————— 38s 20ms/step - accuracy: 0.9756 - loss: 0.0673
<keras.src.callbacks.history.History at 0x7ecefda4e570>

```

```

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
loss_train, accuracy_train = model.evaluate(X_train, y_train, verbose=0)

```

```

print(f'Test loss: {loss:.4f}')
print(f'Test accuracy: {accuracy:.4f}')

```

```

Test loss: 0.4748
Test accuracy: 0.9145

```

```

softmax_model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(10, activation='softmax')
])

```

```
softmax_model.compile(optimizer='adam',
                      loss='sparse_categorical_crossentropy',
                      metrics=['accuracy'])

softmax_model.fit(X_train, y_train, epochs=10, batch_size=128, verbose=1)
softmax_loss, softmax_accuracy = softmax_model.evaluate(X_test, y_test, verbose=1)
softmax_train_loss, softmax_train_accuracy = softmax_model.evaluate(X_train, y_train, verbose=1)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_d
super().__init__(**kwargs)
Epoch 1/10
469/469 ━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.6643 - loss: 1.0256
Epoch 2/10
469/469 ━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8249 - loss: 0.5298
Epoch 3/10
469/469 ━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8383 - loss: 0.4849
Epoch 4/10
469/469 ━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8459 - loss: 0.4515
Epoch 5/10
469/469 ━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.8460 - loss: 0.4488
Epoch 6/10
469/469 ━━━━━━━━━━━ 4s 7ms/step - accuracy: 0.8520 - loss: 0.4350
Epoch 7/10
469/469 ━━━━━━━━━━━ 4s 8ms/step - accuracy: 0.8543 - loss: 0.4240
Epoch 8/10
469/469 ━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8573 - loss: 0.4182
Epoch 9/10
469/469 ━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.8576 - loss: 0.4109
Epoch 10/10
469/469 ━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8600 - loss: 0.4044
313/313 ━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8493 - loss: 0.4378
1875/1875 ━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.8628 - loss: 0.4010
```

```
mlp_model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(256, activation='relu'),
    Dense(10, activation='softmax')
])

mlp_model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

mlp_model.fit(X_train, y_train, epochs=10, batch_size=128, verbose=1)
mlp_loss, mlp_accuracy = mlp_model.evaluate(X_test, y_test, verbose=1)
mlp_train_loss, mlp_train_accuracy = mlp_model.evaluate(X_train, y_train, verbose=1)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_d
super().__init__(**kwargs)
Epoch 1/10
469/469 ━━━━━━━━━━━ 4s 7ms/step - accuracy: 0.7680 - loss: 0.6818
Epoch 2/10
469/469 ━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.8560 - loss: 0.4058
Epoch 3/10
469/469 ━━━━━━━━━━━ 6s 13ms/step - accuracy: 0.8704 - loss: 0.3618
Epoch 4/10
469/469 ━━━━━━━━━━━ 8s 17ms/step - accuracy: 0.8796 - loss: 0.3288
Epoch 5/10
469/469 ━━━━━━━━━━━ 7s 10ms/step - accuracy: 0.8925 - loss: 0.3012
Epoch 6/10
469/469 ━━━━━━━━━━━ 6s 13ms/step - accuracy: 0.8950 - loss: 0.2883
Epoch 7/10
469/469 ━━━━━━━━━━━ 10s 12ms/step - accuracy: 0.8958 - loss: 0.2818
Epoch 8/10
469/469 ━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9032 - loss: 0.2596
Epoch 9/10
469/469 ━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9075 - loss: 0.2512
Epoch 10/10
469/469 ━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9148 - loss: 0.2360
313/313 ━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8782 - loss: 0.3462
1875/1875 ━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.9133 - loss: 0.2348
```

5) Ghi kết quả vào bảng

```
results = pd.DataFrame({
```

```
'(X_train, y_train)': [softmax_train_accuracy, mlp_train_accuracy, accuracy_train],  
'(X_test, y_test)': [softmax_accuracy, mlp_accuracy, accuracy]  
, index=['Softmax', 'MLP', 'CNN'])  
  
print(results)
```

	(X_train, y_train)	(X_test, y_test)
Softmax	0.862967	0.8439
MLP	0.912117	0.8808
CNN	0.995683	0.9145